JAVA INHERITANCE

INHERITANCE

- Inheritance is pure object oriented programming technique.
- Inheritance is a process of creating a new class from old class.
- Constructing / Deriving a class from an existing class.
- New class : Child class / Sub class / Derived class
- Old class : Parent class / Super class / Base class

Implementation

- In java, inheritance is implemented in two ways. They are:
    1. Inheriting classes (Extending classes)
    2. Implementing interfaces (Implementing interfaces)

Characteristics of Inheritance

1. A class can't inherit more than one super class at a time.
2. A class can inherit more than one interface at a time.
3. Sub class can access only the non-private members of the super class.
4. **All the methods declared within super interface must be defined with public modifier in sub class**.

Super Class	:	Most general class
Sub Class	:	Most dependent class.

Construction of new class (sub class)

- Java provides two options for creating a new class (sub class) from an existing class. They are:
    1. Deriving a sub class from super class
    2. Deriving a sub class from super interface.

Syntax of Sub class

1. Deriving a sub class from super class

```
class <sub-class-name> extends <super-class-name>
{
        // variable decl.
        // method decl.
}
```

Example

```
class sample
{
        ...
}
class test extends sample
{
        void disp()
        {
                …
        }
}
```

extends keyword

- This reserved keyword is used to inherit the copy of super class's properties. The properties can be non-private members of super class (Ex. variables, methods, etc, ...)
- It is an important to note that, it is used to permit only one super class at a time.

2. Deriving a sub class from super interface

Syntax

```
class <sub-class-name> implements <interface-name>
{
        // variable decl.
        // method decl.
}
```

Example

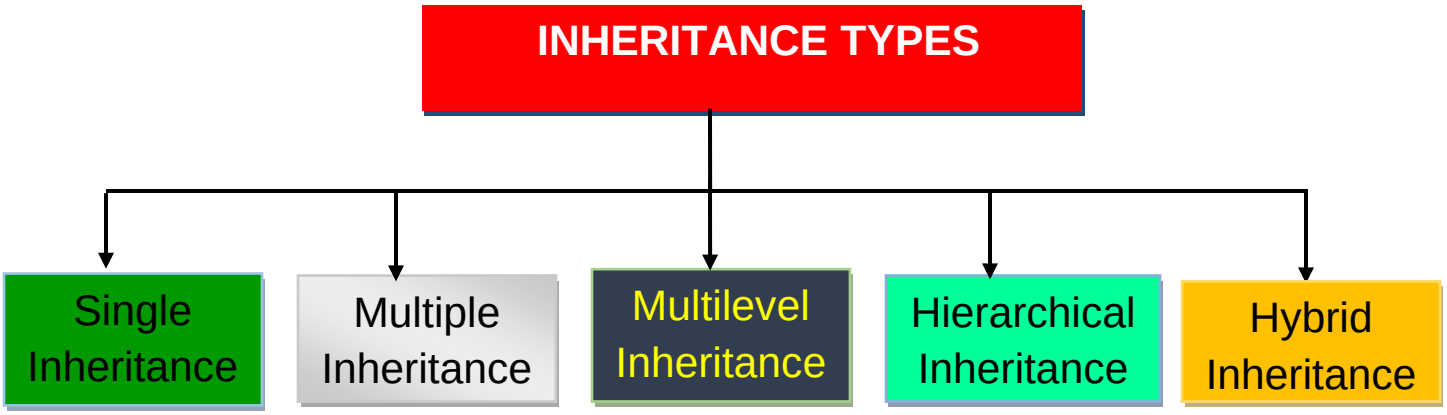
```
interface Icalc                 // super interface
{
        void add();             // method declaration
}
class test extends Icalc
{
        public void add()       // method definition
        {
                …
        }
}
```

implements keyword

- This reserved keyword is used to inherit the copy of super interface 's properties.
- It is used to allow 'n' number of interfaces.

TYPES OF INHERITANCE

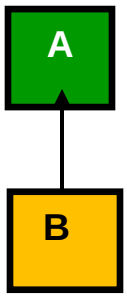
- In java, inheritance is classified as five types. They are
    1. Single Inheritance
    2. Multiple Inheritance
    3. Multilevel Inheritance
    4. Hierarchical Inheritance
    5. Hybrid Inheritance.



Single Inheritance	:	Only one super class & sub class.
Multiple Inheritance	:	Several Super classes.
Multilevel Inheritance	:	Derived from a sub class
Hierarchical Inheritance	:	Only one super class & Several Sub classes
Hybrid Inheritance	:	involving at least one form of inheritance.

1. SINGLE INHERITANCE

- Creating a new class (sub class) from **only one super class**.
- Like 1-1 mapping.



A← Super Class

B← Sub Class.

Fig: Single Inheritance

1. EXAMPLE OF SINGLE INHERITANCE

(B.java)

1. SOURCE CODE

```
// super class definition
class A
{
        int x=25;
        void print_a()
        {
                System.out.println("x = "+x);
        }
}
// sub class definition
class B extends A          // inherit the properties of super class A
{
    int y=50;
// own method of sub class
    void print_b()
    {
// calling super class method
        print_a();
```

```
        System.out.println("y = "+y);
    }
// own method of sub class
    void sum()
    {
        int s=x+y;
        System.out.println("Sum = "+s);
    }
}
// separate main class
public class Single {
    public static void main(String args[])
    {
        System.out.println("============================");
        System.out.println("\t\tSingle Inheritance");
        System.out.println("============================");
// sub class (derived) class object creation
        B obj=new B();
// calling super and sub class instance methods using object
        obj.print_b();
        obj.sum();
    }
}
```

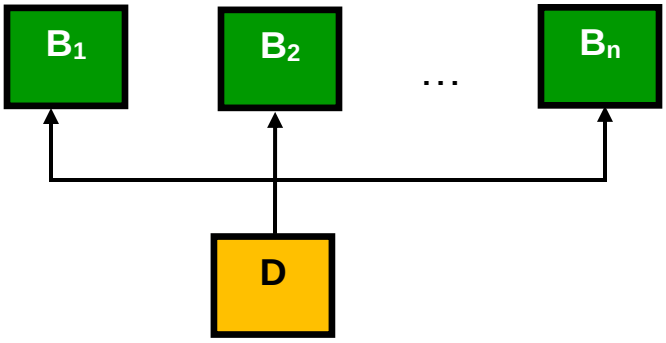
2. OUTPUT

Output - WT_Class_Object (run)

```
run:
=====================================================
                  Single Inheritance
=====================================================
x = 25
y = 50
Sum = 75
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. MULTIPLE INHERITANCE

- The process of creating a new class from more than one super class (several super classes) is called as multiple inheritance.
- Like 1-many.



Where,

$B_1, B_2 .. B_n$ ← Super Classes D ← Sub Class

Fig: Multiple Inheritance

NOTABLE POINTS

- C++ gives direct support for multiple inheritances. But in java, **it is not possible to inherit more than one super class at a time**.
- Java does **not directly support multiple inheritances**. But it gives indirect support of multiple inheritances. This is implemented with help of interfaces in java.

INTERFACE

- It is a new concept of java. It gives the solution for multiple inheritances in both Java / C#.NET
- An interface is basically a kind of class. But it contains abstract methods and final fields by default
- It contains only method declaration. We must provide the method implementation in a sub class.
- It is not possible to create an object for interface in java. With help of sub class object, we can call the interface methods.
- By default, all the methods declared inside an interface are abstract methods even the abstract keyword is not used and by default.

- By default, all the methods in an interface are public.
- By default, all the variables defined inside an interface are final variables (constant variables) even final keyword is left.
- Implements keyword is used to inherit the properties of interface.

DIFFERENCE BETWEEN CLASS AND INTERFACE

S.N	Class	Interface
1.	User defined type. It has member's declaration & definitions.	Block of code like classes. It contains only method declaration. It has no method definition
2.	All the methods & variables are friendly by default.	All the methods are **public** by default. All the variables **final** by default.
3.	Creating an object for a class **is possible**	Creating an object for an interface is **not possible** directly.
4.	It supports declaration and definition.	It supports declaration only. Method implementation must be given to derived class using public modifier.
5.	It supports constructors	It does not support constructors

Syntax of Interface

```
interface <interface-name>
{
        // constant variable definition
        // method declarations only (no definition)
}
```

Example of Interface

```
interface Circle
{
        float PI=3.14f;
        void area();
}
```

Interface Member Access

- Java provides two options for calling the methods of interface
    1. Using interface object through object alias (assigning sub class object to interface object)
    2. Using sub class object.

Interface Object Creation

- Creating an object for interface is not possible directly
- But indirectly we can create an object by just assigning the object of sub class to interface object variable.

Syntax

```
<interface-name> obj=<sub-class object>
```

Example

```
interface Calc
{
	void add();
}
public class SubImp implements Calc
{
	public void add()
	{
	}
	public static void main(String[] args)
	{
// object creation for sub class
		SubImp obj=new SubImp();
// interface object creation
		Calc cc=obj;
// call interface methods using interface object (not sub class object)
		cc.add();
	}
}
```

2. EXAMPLE OF MULTIPLE INHERITANCE

(Multiple.java)

1. SOURCE CODE

```
// super interface 1
interface Person
{
    String name="Sachin";            // constant variables
    final int age=25;
    void person_info();              // method declaration
}
// super interface 2
interface Employee
{
    int id=233;
    String cmp="Google";
    void emp_info();
}
// sub class
class D implements Person, Employee
{
    double income=95000.45638;
// super interface 1 method
    public void person_info()
    {
        System.out.println("Name \t: "+name);
        System.out.println("Age \t: "+age);
    }
// super interface 2 method
    public void emp_info()
    {
        System.out.println("Id \t: "+id);
        System.out.println("Company\t: "+cmp);
```

inherits two super interfaces **Person, Employee**.

```
    }
// own method of sub class
    void disp()
    {
        person_info();
        emp_info();
        System.out.printf("Income\t: %8.2f\n",income);
    }
}
// separate main class
public class Multiple {
    public static void main(String a[])
    {
         System.out.println("================================");
         System.out.println("\tMultiple Inheritance");
         System.out.println("================================");
// object creation for sub class
         D obj=new D();
         obj.disp();
    }
}
```

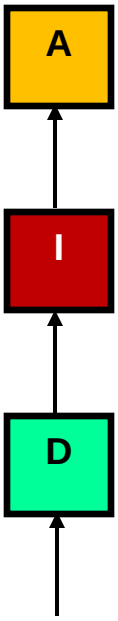
2. OUTPUT

Output - WT_Class_Object (run)

```
run:
==============================================
        Multiple Inheritance
==============================================
Name    : Sachin
Age     : 25
Id      : 233
Company : Google
Income  : 95000.46
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. MULTILEVEL INHERITANCE

- The process of creating a new class from another derived class or sub class (already inherited sub-class) is called as multilevel inheritance.
- **Intermediate class**
  - If a class acts as super class on one side and sub class on another side is called as intermediate class.



N Level

Fig: Multilevel Inheritance

Where,

- Class A is called as super class
- Class I is called as intermediate class
- Class D is called as derived class.

3. EXAMPLE OF MULTILEVEL INHERITANCE

(Multilevel.java)

1. SOURCE CODE

```
// super class definition
class P
{
    int a=25;                    // default modifier
    void printP()
    {
        System.out.println("A = "+a);
    }
}
// sub class (Intermediate class): inherit one copy of class P
class Q extends P
{
    int b=25;                    // default modifier


    void printQ()
    {
        System.out.println("B = "+b);
    }
}
// sub class: inherit one copy of Q class using extends keyword
class R extends Q
{
    int c=50;                    // default modifier
    void printR()
    {
        System.out.println("C = "+c);
    }
```

Intermediate class

```
    void sum()
    {
        int res=a+b+c;
        System.out.println("Sum = "+res);
    }
}
// separate Main class
public class Multilevel {
    public static void main(String a[])
    {
        System.out.println("==============================");
        System.out.println("\tMultilevel Inheritance");
        System.out.println("============================");
// object creation for sub class
        R obj=new R();
// call own method & super class methods using sub class object
        obj.printP();
        obj.printQ();
        obj.printR();
        obj.sum();
        System.out.println("-----------------------------------------");
    }
}
```

2. OUTPUT

Output - WT_Class_Object (run)

```
run:
==============================================
        Multilevel Inheritance
==============================================
A = 25
B = 25
C = 50
Sum = 100
----------------------------------------------
BUILD SUCCESSFUL (total time: 0 seconds)
```

4. HIERARCHICAL INHERITANCE

- The process of creating several new classes from only one super class / base class is called as hierarchical inheritance.
- Like many to 1 mapping.

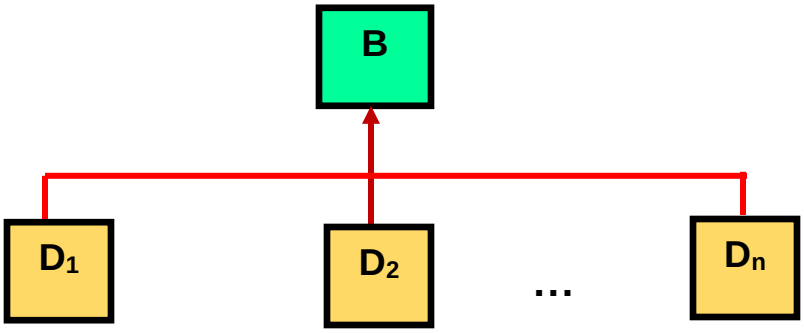


Fig: Hierarchical Inheritance

Where,

- **D_1, D_2, …D_n** are list of sub classes
- **B** is a super class.

4. EXAMPLE OF HIERARCHICAL INHERITANCE

(Hierarchical.java)

1. SOURCE CODE

```
// super class definition
class S
{
    int a=35;
    void printS()
    {
        System.out.println("a ="+a);
    }
}
```

```
// subclass 1: inherit the copy of S(super class)
class Sub1 extends S
{
    int d1=50;

    void printS1()
    {
        printS();
        System.out.println("d1 ="+d1);
    }
}
// subclass 2: inherit the copy of S (super class)
class Sub2 extends S
{
    int d2=75;

void printS2()
    {
        printS();
        System.out.println("d2 ="+d2);
    }
}
// subclass 3: inherit the copy of S (super class)
class Sub3 extends S
{
    int d3=150;
    void printS3()
    {
        printS();
        System.out.println("d3 ="+d3);
    }
}
```

```
// separate main class
public class Hierarchical
{
    static public void main(String[] args)
    {
        System.out.println("==========================");
        System.out.println("\tHierarchical Inheritance");
        System.out.println("==========================");
// sub class 1 object creation
        Sub1 obj1=new Sub1();
        System.out.println("--------------Sub Class 1------------------");
        obj1.printS1();
// sub class 2 object creation
        Sub2 obj2=new Sub2();
        System.out.println("--------------Sub Class 2------------------");
        obj2.printS2();
// sub class 3 object creation
        Sub3 obj3=new Sub3();
        System.out.println("--------------Sub Class 3------------------");
        obj3.printS3();
    }
}
```

2. OUTPUT

Output - WT_Class_Object (run)

```
run:
==============================================
        Hierarchical Inheritance
==============================================
--------------Sub Class 1-------------------
a =35
d1 =50
--------------Sub Class 2-------------------
a =35
d2 =75
--------------Sub Class 3-------------------
a =35
d3 =150
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. HYBRID INHERITANCE

- Process of creating new class involving **at least one form of inheritance** (single or multiple or multilevel or hierarchical) and **one or more number of base classes (super classes)** is called as hybrid inheritance.

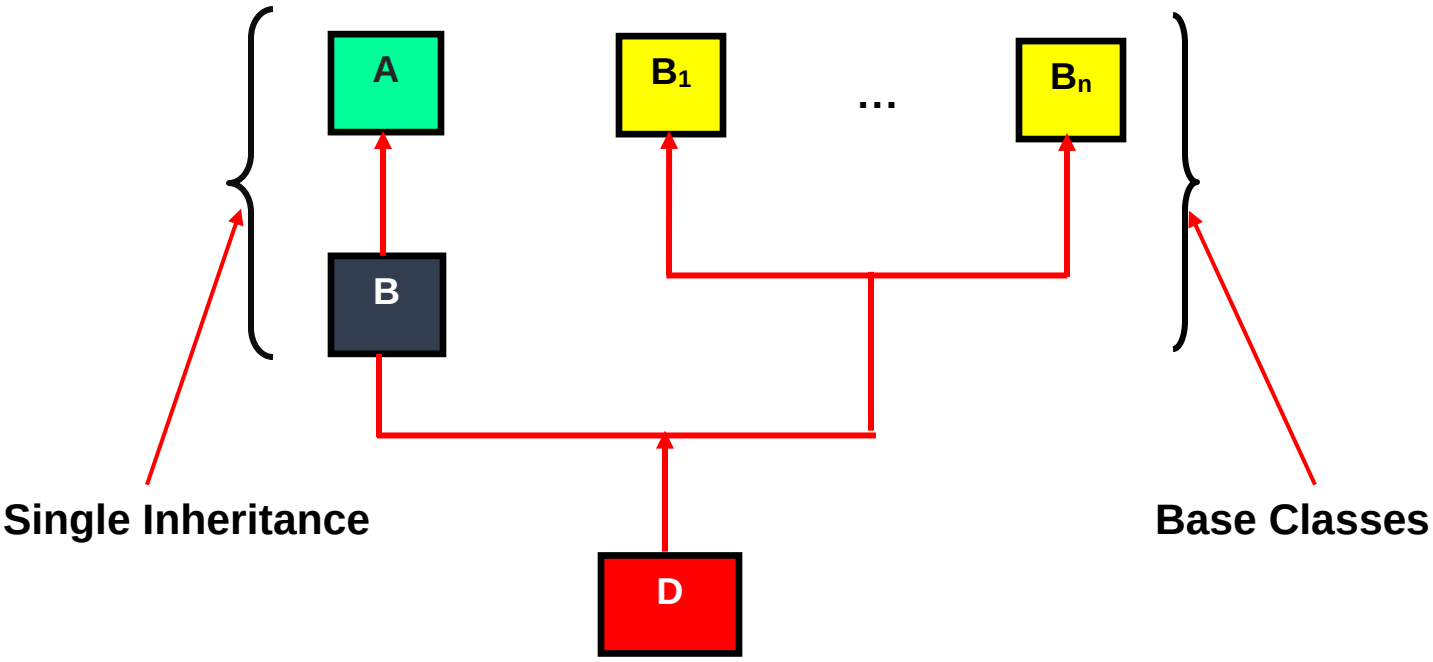


Figure. Hybrid Inheritance

Where,

- A→ B is a **single inheritance**
- B_1, … B_n are super classes
- D is a sub class.

5. EXAMPLE OF HYBRID INHERITANCE

(Hybrid.java)

1. SOURCE CODE

```
// super class definition
class X
{
    int a=191;
    void printX()
    {
        System.out.println("a = "+a);
    }
}
// single Inheritance
class SI extends X
{
    int b=283;
    void print_SI()
    {
        printX();                    // call the base class method
        System.out.println("b = "+b);
    }
}
// super interface (super class)
interface SS
{
    int c=100;
    void print_SS();
}
// new sub class(hybrid class)
class SBI extends SI implements SS
{
    int d=308;
```

```
    public void print_SS()
    {
        System.out.println("c = "+c);
    }
    // own method 1 of sub class
    void print_SBI()
    {
        print_SS();                    // call the interface method
        System.out.println("d = "+d);
    }
    // own method 2 of sub class
    void sum()
    {
        int res=a+b+c+d;
        System.out.println("Sum = "+res);
    }
}
// separate main class
public class Hybrid
{
    public static void main(String[] args)
    {
         System.out.println("============================");
         System.out.println("\tHybrid Inheritance");
         System.out.println("============================");
// object creation for sub class
         SBI obj=new SBI();
// calling instance methods using object
         obj.print_SI();
         obj.print_SBI();
         obj.sum();
    }
}
```

2. OUTPUT

Output - WT_Class_Object (run)

```
run:
==============================================
        Hybrid Inheritance
==============================================
a = 191
b = 283
c = 100
d = 308
Sum = 882
BUILD SUCCESSFUL (total time: 0 seconds)
```